Retrieval-Augmented Generation (RAG)

An overview of the concept, architecture, and trade-offs

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines a large language model (LLM) with an external knowledge retrieval step. Instead of relying only on knowledge learned during training, a RAG system searches a document store or index at query time, retrieves the passages most relevant to the user's question, and feeds them to the LLM as additional context before it generates an answer.

This allows an LLM to answer questions about private, proprietary, or frequently changing information — such as internal company documents, product manuals, or news — without needing to retrain or fine-tune the underlying model.

Why Use RAG?

- **Up-to-date and private knowledge:** answers can reflect internal or recent data the model never saw during training.
- **Reduced hallucination:** grounding responses in retrieved source text makes answers more factual and verifiable.
- **Traceability:** responses can cite specific source documents or passages, aiding trust and auditability.
- **Lower cost than fine-tuning:** updating a search index is far cheaper and faster than retraining a model.
- **Domain adaptability:** the same base model can serve many use cases simply by swapping the underlying knowledge base.

Core Components of a RAG Pipeline

1. **Document ingestion:** source files (PDFs, docs, web pages, etc.) are collected and split into smaller, semantically coherent chunks.
2. **Embedding & indexing:** each chunk is converted into a vector embedding and stored in a search index (vector, keyword, or hybrid).
3. **Retrieval:** at query time, the user's question is embedded and used to find the most semantically relevant chunks in the index.
4. **Augmentation:** the retrieved chunks are inserted into the prompt sent to the LLM alongside the original question.
5. **Generation:** the LLM produces a final answer grounded in the retrieved context, often with citations back to source documents.

Common RAG Architectures

- **Naive / basic RAG:** a single retrieval pass followed directly by generation. Simple and fast, but can miss context if the first retrieval isn't precise.
- **Hybrid search RAG:** combines keyword (BM25) search with vector similarity search to balance exact-term matching with semantic understanding.
- **Re-ranking RAG:** retrieves a larger candidate set, then uses a secondary re-ranking model to reorder results by relevance before passing them to the LLM.
- **Agentic / multi-step RAG:** the model can issue multiple retrieval queries, reason over intermediate results, and refine its search before producing a final answer.
- **Graph RAG:** retrieves from a knowledge graph in addition to (or instead of) unstructured text, useful for highly relational data.

Key Design Considerations

1. **Chunking strategy:** chunk size and overlap significantly affect retrieval quality — too small loses context, too large dilutes relevance.
2. **Embedding model choice:** affects how well semantically similar text is matched, especially across domains or languages.
3. **Retrieval count (top-k):** too few chunks risks missing key information; too many adds noise and increases token cost.
4. **Freshness:** the index must be kept in sync with source documents, especially for frequently changing content.
5. **Evaluation:** RAG systems are typically evaluated on retrieval quality (precision/recall) and answer quality (faithfulness, relevance, groundedness).

Common Use Cases

- Enterprise knowledge assistants (internal wikis, policies, support docs)
- Customer support chatbots grounded in product documentation
- Legal and compliance document search and summarization
- Research assistants that cite academic papers or reports
- Coding assistants grounded in a specific codebase or API documentation

---

Summary: RAG augments an LLM's fixed training knowledge with a live retrieval step, improving factual accuracy, enabling use of private or current data, and providing traceable citations — at the cost of added architectural complexity around chunking, indexing, and retrieval tuning.